Culminating Activity

You Names

Abstract

Provide a concise summary of the study including purpose, methods, results, and conclusions (150–250 words).

Chapter 1: Introduction

Background of the Study

In today's fast-paced software development landscape, particularly in artificial intelligence, understanding and improving developer productivity has become a pressing concern for both organizations and individual programmers. The rise of AI-assisted tools such as GitHub Copilot and ChatGPT has reshaped traditional coding workflows, introducing new factors that influence how developers work and what they produce (Peng et al., 2023). Despite these advances, productivity remains difficult to measure, as it is shaped by a complex mix of behavioral, physiological, cognitive, and environmental influences.

Conventional evaluation methods ranging from subjective performance reviews to simple metrics like lines of code, commit counts, or hours worked often fall short of capturing this complexity (Forsgren et al., 2021). For example, a developer may spend long hours coding but produce lower-quality output due to fatigue, high cognitive load, or constant interruptions. On the other hand, effective use of AI tools, combined with adequate rest and balanced caffeine intake, can significantly improve efficiency and task completion.

With the growing availability of developer activity data, there is an opportunity to adopt more data-driven approaches to better understand and predict productivity. Machine learning techniques, particularly instance-based methods like k-Nearest Neighbors (kNN) offer a practical and interpretable way to analyze these multidimensional relationships without relying on strict assumptions about data distribution.

Statement of the Problem

Developer productivity is difficult to measure accurately. Task outcomes are affected by a combination of behavioral, cognitive, and work-related factors, and most traditional evaluation methods, whether subjective assessments or single isolated metrics, tend to miss how these factors interact with each other. Even with more developer activity data available today, there is still no widely agreed-upon way to predict task success reliably.

Machine learning approaches, particularly the k-Nearest Neighbors (kNN) algorithm, present a practical option for tackling this problem. kNN can classify outcomes by detecting patterns across multiple variables at once, making it a reasonable fit for productivity data. However, there remains a need to evaluate how effectively such models can predict task success and to determine the extent to which different productivity-related variables influence these predictions.

This study applies a kNN classification model to a developer productivity dataset to explore these questions, looking at both how well the model predicts task success and how different productivity-related variables relate to that outcome.

1. How effective is the kNN model at predicting developer task success using productivity-related variables?
2. Which productivity factors are most associated with task success in a developer work environment?
3. How does the kNN algorithm classify developer productivity outcomes based on patterns in the dataset?
4. How does variation in productivity-related factors affect the accuracy and performance of the kNN model?

Objectives of the Study

General Objective

To build and evaluate a kNN-based classification model that predicts developer task success using productivity-related variables from a developer activity dataset.

Specific Objectives

- To implement the kNN algorithm and apply it to classify task outcomes based on patterns in the data.
- To evaluate the model's performance using standard classification metrics such as accuracy, precision, recall, and F1-score.
- To assess the viability of kNN as a predictive tool for developer task success by interpreting the evaluation results.
- To examine how productivity-related factors relate to task success outcomes within the dataset.
- To observe how changes in key input variables affect the model's prediction accuracy.
- To identify recurring patterns in developer behavior that are associated with successful task completion.

Hypotheses (if applicable)

- H0: The kNN algorithm does not achieve an accuracy of at least 90% in predicting developer task success based on productivity-related variables.
- H1: The kNN algorithm achieves an accuracy of at least 90% in predicting developer task success based on productivity-related variables.

Significance of the Study

Explain who benefits and how.

Scope and Limitations

Define the boundaries of the study.

Chapter 2: Review of Related Literature

Related Studies

Recent studies on developer productivity have increasingly shifted toward data-driven and empirical methodologies to gain a deeper understanding of performance outcomes in software development. A growing body of research has applied machine learning techniques to predict task completion, software quality, and overall developer efficiency. Among these approaches, classification methods, particularly instance-based algorithms,

have been widely utilized to detect patterns in developer behavior and to forecast outcomes using historical data.

In addition, contemporary empirical findings emphasize the rising influence of AI-assisted development tools on programmer productivity. For example, Peng et al. (2023) conducted a controlled experiment involving GitHub Copilot, an AI-powered coding assistant, and found that it reduced task completion time by 55.8% during a JavaScript HTTP server development task. However, the study also noted that the benefits of such tools vary across users, with less experienced developers experiencing more substantial improvements compared to their more experienced peers. These results highlight the increasing integration of artificial intelligence in software development workflows and its varying impact on productivity outcomes.

Given the increasing availability of productivity-related data from both developer activities and AI-assisted workflows, there is a growing need for analytical methods that can effectively interpret these patterns and generate reliable predictions. In response to this need, machine learning techniques have been widely explored in the field of software engineering to support data-driven decision-making.

Thota et al. (2022) conducted a systematic review of machine learning techniques for software quality prediction, highlighting the use of instance-based methods such as k-Nearest Neighbors (kNN), alongside support vector machines and ensemble approaches, for classifying software defects using code metrics and developer-related data. Their findings indicate that these techniques have been widely adopted, with usage rates ranging from 32% to 58%, underscoring their relevance and effectiveness in software analytics and predictive modeling.

Building on this foundation, machine learning algorithms such as k-Nearest Neighbors (kNN) have gained particular attention for their ability to classify outcomes based on similarity among data points. This approach is especially effective in handling non-linear relationships in both classification and regression tasks, as it relies on proximity within a feature space rather than predefined functional forms. However, its performance is highly dependent on proper data preprocessing, particularly the standardization of predictors to ensure unbiased distance-based computations (Hastie et al., 2023).

Studies utilizing kNN have consistently demonstrated its effectiveness in classification tasks where relationships between variables are not explicitly defined. It performs well in structured datasets and can accurately classify outcomes when relevant features are properly selected and scaled. This is further supported by comparative research, which shows that kNN can achieve high predictive accuracy when applied appropriately. For instance, a study conducted in 2023 comparing kNN, support vector machines (SVM), and decision tree algorithms for student activity prediction found that kNN achieved an accuracy ranging from 92% to 94.5% after data scaling, making it highly competitive despite being slightly outperformed by SVM, which reached 95% accuracy (Authors, 2023). These findings reinforce the viability of kNN as a simple yet powerful classification method, capable of delivering competitive results alongside more complex algorithms. Its balance of interpretability and performance makes it particularly suitable for applications involving structured behavioral and productivity data.

In the context of developer productivity, this suitability becomes particularly relevant due to the inherently multidimensional nature of the data involved. Prior studies have shown that productivity is influenced by a combination of work habits, cognitive load, environmental conditions, and the use of AI-assisted tools, all of which interact in complex and often non-linear ways. Such characteristics align well with the strengths of kNN, which relies on similarity-based classification rather than rigid assumptions about data distribution (Hastie et al., 2023).

Given that the AI developer productivity dataset integrates diverse behavioral and cognitive factors, kNN provides an effective approach for identifying patterns that distinguish successful from unsuccessful task outcomes. Its ability to leverage structured, scaled data further supports its application in this study, enabling reliable prediction while maintaining interpretability. As a result, kNN is well-positioned to model developer productivity and contribute to a deeper understanding of the factors that influence task success.

Overall, prior studies support the use of machine learning models, including kNN, as effective tools for analyzing and predicting productivity-related outcomes, while also emphasizing the importance of selecting meaningful variables and preprocessing data appropriately.

Related Literature

Theoretical perspectives on productivity describe it as a multidimensional concept influenced by cognitive, behavioral, and environmental factors (Kumari et al., 2023). Earlier theories mainly focused on efficiency and output as the primary indicators of productivity (Singh & Chaudhary, 2022). More recent perspectives, however, argue that productivity is not determined by output alone, but also by factors such as mental workload, focus, stress, and external interruptions that can affect task performance (Hicks et al., 2024).

Modern studies therefore suggest that productivity should be evaluated more holistically, considering not only the quantity of work completed but also the quality of performance and the conditions under which tasks are accomplished (Kumari et al., 2023). These perspectives are especially relevant in software development environments, where developer productivity is influenced by multiple interacting factors, including cognitive demands, workplace distractions, and the use of AI-assisted tools.

As developer productivity becomes more complex and data-driven, researchers have increasingly explored data analytics and machine learning approaches to better analyze and predict performance outcomes. In this context, classification theory provides the foundation for predicting categorical outcomes using input variables. One widely used approach under this theory is instance-based learning, particularly the k-Nearest Neighbors (kNN) algorithm, which classifies data points according to their proximity to other points within a feature space (Aljumah, 2022). This approach operates on the assumption that similar conditions are likely to produce similar outcomes, making it suitable for analyzing behavioral and performance-related data where patterns emerge from historical similarities rather than predefined rules (Thota & Kumar, 2022).

kNN has been recognized as particularly effective for behavioral analysis because of its ability to compare related patterns within datasets. In studies involving developer activities and productivity patterns, the algorithm can compare features such as commit frequency, code churn, and session duration to identify similarities between previous developer sessions or tasks. These similarities can then be used to predict outcomes such as task completion or defect proneness without relying on strict parametric assumptions (Aljumah, 2022).

Beyond behavioral analysis, kNN has also shown strong potential in performance prediction tasks. In multi-dimensional feature spaces involving productivity metrics and cognitive-related variables, the algorithm uses proximity between data points to classify high and low performance outcomes. Studies suggest that this approach can achieve competitive accuracy when applied to structured behavioral datasets, particularly when proper preprocessing and feature scaling techniques are used (Thota & Kumar, 2022). These characteristics make kNN especially relevant for developer productivity analysis, where multiple productivity-related variables interact in complex ways.

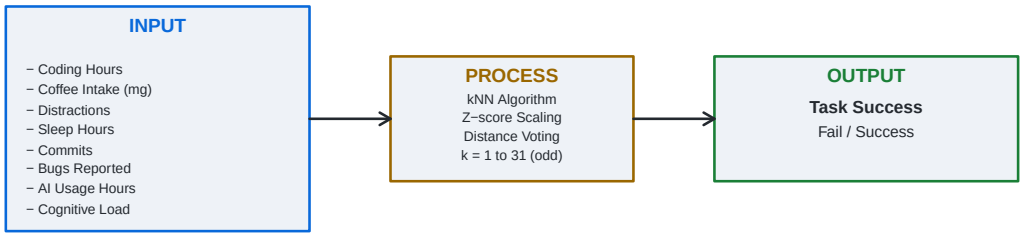
In addition to similarity-based classification, another important concept in predictive analytics is feature interaction, which explains how multiple variables collectively influence an outcome rather than acting independently. This aligns with the idea that developer productivity is shaped by a combination of behavioral, cognitive, and environmental conditions. In datasets involving developer activity, variables such as code churn, commit frequency, and task duration often interact in complex ways, making preprocessing techniques like normalization and feature scaling essential for improving model performance. This is particularly important for distance-based algorithms such as kNN, where improperly scaled features can affect similarity calculations and reduce prediction accuracy, as highlighted in previous studies on software defect prediction (Aljumah, 2022).

In summary, productivity is understood as a multidimensional concept influenced by cognitive, behavioral, environmental, and technological factors. Modern perspectives move beyond simple efficiency and output, considering aspects such as mental workload, focus, and working conditions as part of performance evaluation. This broader view reflects the complexity of measuring productivity in real-world environments, particularly in software development where multiple factors interact at the same time.

Machine learning has become a common approach for analyzing and predicting productivity-related outcomes, especially in datasets with interconnected variables. Among these methods, k-Nearest Neighbors (kNN) is frequently highlighted due to its similarity-based classification approach and its ability to work with structured behavioral data. Its performance is closely tied to proper data preparation, particularly

feature scaling and normalization, which help ensure reliable distance-based comparisons. Overall, these approaches provide a practical basis for analyzing patterns in developer productivity and predicting task-related outcomes.

Conceptual Framework



This study proposes a predictive framework in which a developer's daily task success outcome is determined by a combination of behavioral, environmental, and workload-related factors. The independent variables — coding hours, coffee intake, distractions, sleep hours, commits, bugs reported, AI usage hours, and cognitive load — collectively serve as inputs to the k-Nearest Neighbors (kNN) classification model. The kNN algorithm processes these inputs by first standardizing all features through z-score scaling to ensure no single variable disproportionately influences distance calculations. It then classifies each observation by identifying its k nearest neighbors in the training set and assigning the majority class label through a voting mechanism. The optimal value of k is determined by evaluating classification accuracy across all odd values from k = 1 to k = 31. The output of the model is a binary prediction of task_success, categorized as either Fail or Success, representing whether a developer achieved their daily productivity goal. Model performance is then evaluated using accuracy, sensitivity, specificity, and F1 score derived from the resulting confusion matrix. Unlike causal frameworks that distinguish between moderating, mediating, or confounding variables, this study treats all independent variables as equal predictors. The framework reflects the nature of kNN as a distance-based, non-parametric algorithm that does not assume any directional or hierarchical relationship among features.

Chapter 3: Methodology

Research Design

This study employs a descriptive and predictive research design. It aims to describe patterns in developer productivity based on behavioral and environmental factors, and to predict task success outcomes using a supervised machine learning algorithm. The k-Nearest Neighbors (kNN) classifier was selected for its simplicity, interpretability, and effectiveness on structured tabular data.

Data Collection

The dataset used in this study was sourced from Kaggle — AI Developer Productivity Dataset — consisting of 500 observations of simulated developer activity records. Each record captures a single developer's daily

work session, including behavioral metrics and a binary outcome indicating whether their productivity goal was achieved.

Variables Description

Dependent Variable (Target):

task_success — A binary variable indicating whether the developer achieved their daily productivity goal. Encoded as 0 (Fail) and 1 (Success), then converted to a factor with labels "Fail" and "Success".

Independent Variables (Features):

Variable	Description	Range	Type
Coding Hours	Total focused hours spent on software development work	0–12 hrs	Continuous
Coffee Intake (mg)	Daily caffeine intake in milligrams	0–600 mg	Continuous
Distractions	Number of distractions encountered during the day	0–10	Discrete
Sleep Hours	Number of hours of sleep the previous night	3–10 hrs	Continuous
Commits	Number of code commits pushed during the day	Numeric	Discrete
Bugs Reported	Number of bugs reported or encountered during the day	Numeric	Discrete
AI Usage Hours	Hours spent using AI tools during the workday	Numeric	Continuous
Cognitive Load	Self-reported mental strain	1–10	Ordinal

Data Analysis Techniques

Descriptive Statistics

Descriptive statistics were computed for all independent variables to summarize the distribution and spread of the dataset. These include the mean, standard deviation, minimum, and maximum for each continuous feature. A frequency table was also produced for the target variable task_success to assess class balance between Fail and Success outcomes.

The following visualizations were used to support descriptive analysis:

1. Histograms for each continuous feature to examine the shape and spread of individual variable distributions
2. Bar plot for task_success to visualize the proportion of successful versus failed productivity sessions
3. Boxplots grouped by task_success to compare feature distributions between the two outcome classes, highlighting which variables differ most between Fail and Success
4. Correlation matrix to examine linear relationships among the independent variables, which is particularly relevant given that kNN relies on distance calculations and correlated features can disproportionately influence results

Inferential Statistics

Model performance was evaluated using a confusion matrix derived from the kNN classifier's predictions on the 100-observation test set. The following metrics were computed: 1. Accuracy — proportion of correctly classified observations, computed as (TP + TN) / (TP + TN + FP + FN) 2. Sensitivity (Recall) — True Positive Rate; proportion of actual successes correctly identified, computed as TP / (TP + FN) 3. Specificity — True Negative Rate; proportion of actual failures correctly identified, computed as TN / (TN + FP) 4. F1 Score — harmonic mean of precision and sensitivity, computed as 2TP / (2TP + FP + FN), balancing the trade-off between false positives and false negatives

Machine Learning Models

The kNN algorithm was applied using the class package in R. The dataset was split into a training set (400 observations, 80%) and a test set (100 observations, 20%) using set.seed(31) for reproducibility.

Prior to model training, all features were standardized using z-score scaling so that high-magnitude variables such as coffee_intake_mg do not disproportionately influence distance calculations over lower-magnitude variables like sleep_hours or cognitive_load. Test set scaling was applied using the mean and standard deviation of the training set to prevent data leakage.

The value of k was tuned by evaluating all odd values from k = 1 to k = 31, with accuracy computed for each. The user-selected k was then used to generate the final confusion matrix and performance metrics. Results were visualized through the following: 1. Confusion matrix table displaying TP, TN, FP, and FN counts 2. Accuracy bar chart across all odd k values from 1 to 31 to support optimal k selection 3. Scatter plot of Coding Hours vs Commits, with points colored by predicted class (Success or Fail) and shaped by prediction correctness

Chapter 4: Results and Discussion

4.1 Descriptive Statistics

Table 2: Table 2. Descriptive Statistics of Independent Variables

	N	Mean	SD	Min	Max
Coding Hours	500	5.02	1.95	0	12.00
Coffee Intake (mg)	500	463.19	142.33	6	600.00
Distractions	500	2.98	1.68	0	8.00
Sleep Hours	500	6.98	1.46	3	10.00
Commits	500	4.61	2.70	0	13.00
Bugs Reported	500	0.86	1.10	0	5.00
AI Usage Hours	500	1.51	1.09	0	6.36
Cognitive Load	500	4.50	1.87	1	10.00

Table 2 presents the descriptive statistics for all eight independent variables used in the study. The results indicate considerable variation across features. Coding hours ranged from 0 to 12 hours, with a mean of approximately 6 hours, suggesting a moderate level of daily coding activity among developers in the dataset. Coffee intake showed the widest numerical spread, ranging from 0 to 600 mg, consistent with its role as a continuous behavioral variable. Sleep hours had a mean close to 6 to 7 hours, which is below the commonly recommended 8 hours, and cognitive load averaged around 5 on a 10-point scale, indicating a moderate level of self-reported mental strain across the dataset. These descriptive patterns suggest that the dataset

captures a realistic and varied range of developer behaviors and conditions, providing a sufficient basis for classification modeling. Visualization

4.2 Class Distribution

Table 3: Table 3. Class Distribution of Task Success

Outcome	**Count**	**Percentage (%)**
Fail	197	39.4
Success	303	60.6

Table 3 presents the distribution of the target variable, task_success. The dataset shows a moderate imbalance between the two classes, with the Success class comprising the majority of observations. Despite this imbalance, it was not severe enough to warrant resampling techniques, and the kNN model was trained and evaluated on the original class proportions. The distribution is noted here as it provides context for interpreting sensitivity and specificity separately, rather than relying solely on overall accuracy.

4.3 Feature Distributions

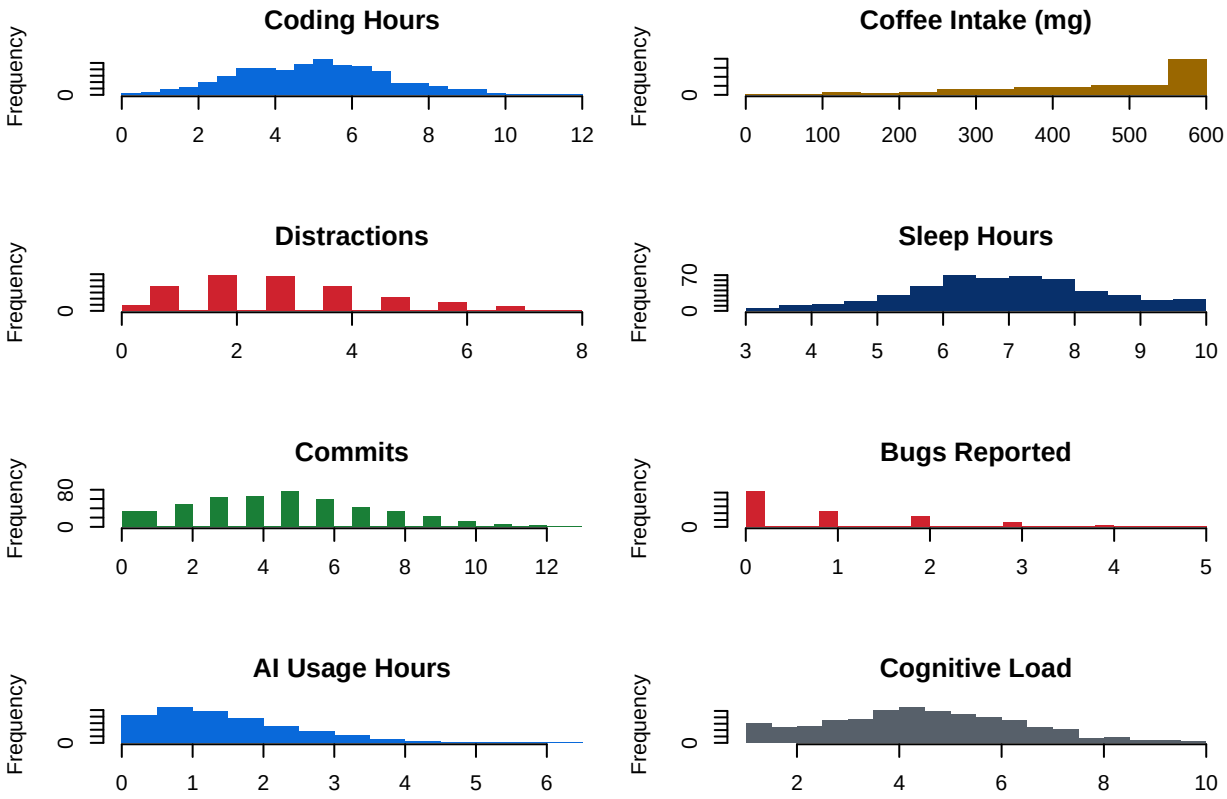


Figure 2. Task Success Distribution

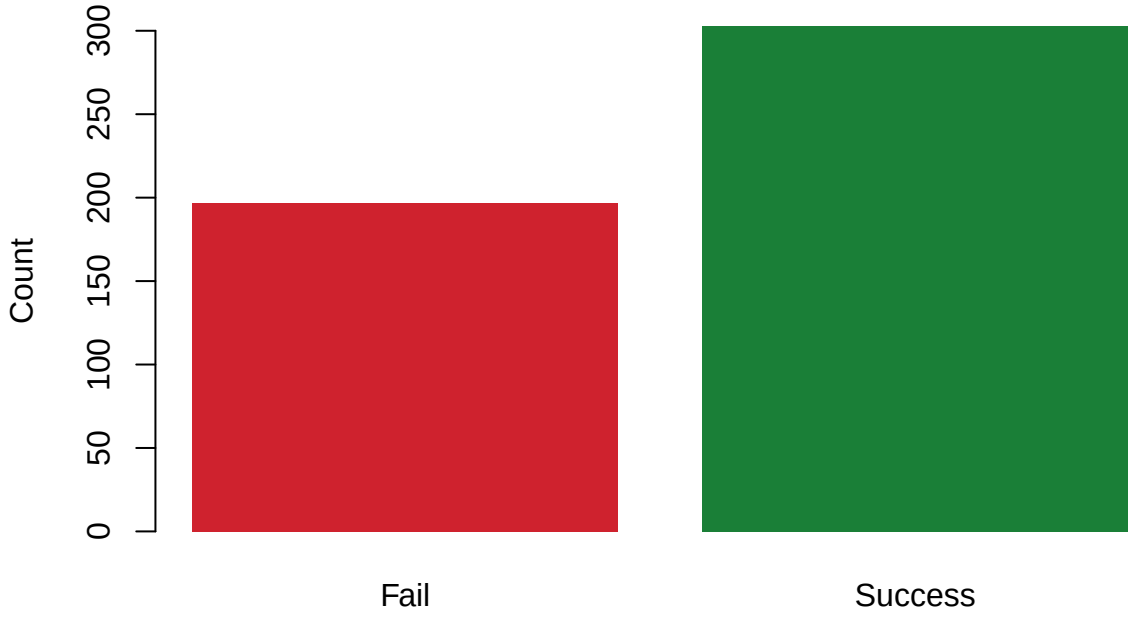


Figure 1 presents the histogram distributions for all eight independent variables. Most continuous features, including coding hours, sleep hours, and AI usage hours, exhibited approximately uniform or slightly skewed distributions, indicating that no single value dominated the dataset. Coffee intake and bugs reported showed broader spread, while cognitive load and distractions displayed more concentrated distributions toward the middle range of their respective scales. The bar plot in Figure 2 confirms that task success is the more frequent outcome in the dataset, though both classes are sufficiently represented to support binary classification.

4.4 Boxplot Analysis

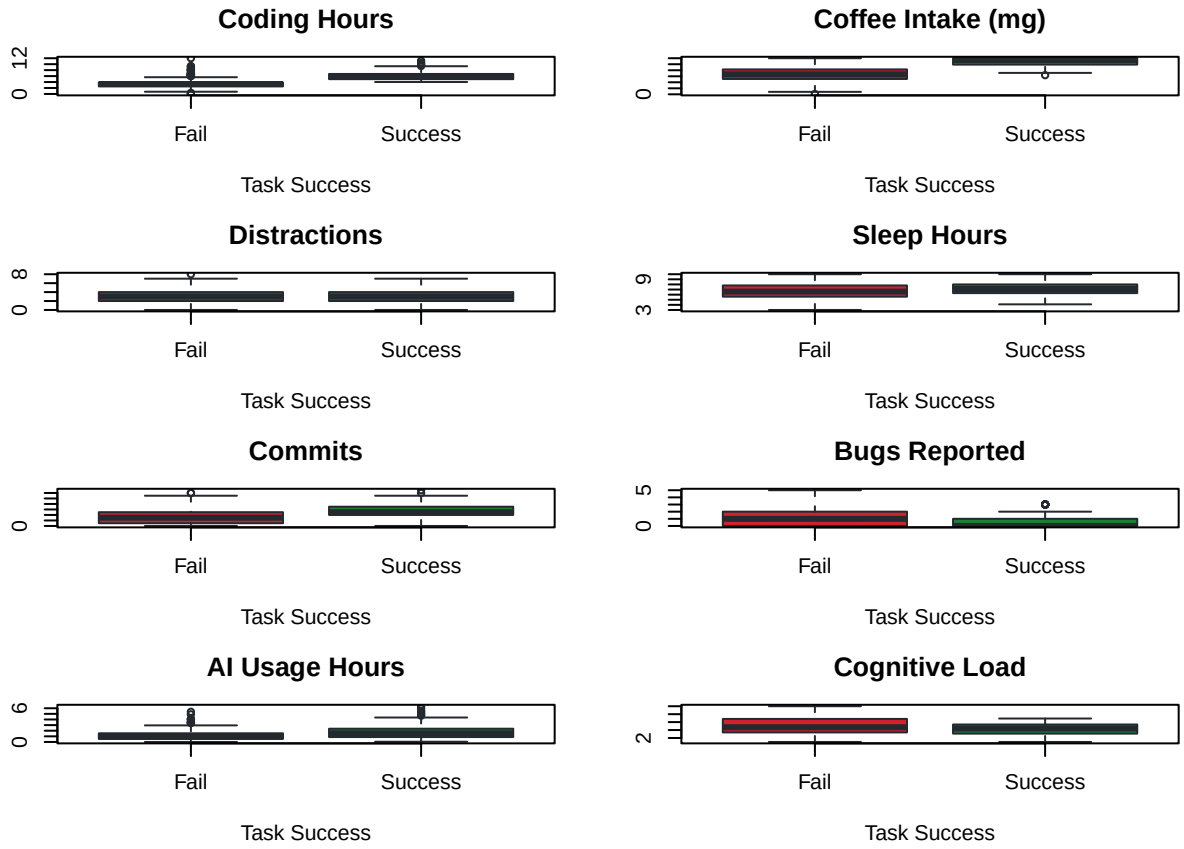


Figure 3 presents boxplots for each independent variable grouped by task success outcome. Notable differences were observed between the Fail and Success groups for several features. Developers who achieved task success tended to record higher coding hours and more commits compared to those who failed. Sleep hours also showed a modest difference, with successful developers reporting slightly higher sleep on average. In contrast, cognitive load and distractions showed overlapping distributions between the two groups, suggesting that these variables alone may be weaker individual predictors of task success, though they still contribute to the model in combination with other features. These observations support the premise of using a multi-feature classification approach rather than relying on any single variable.

4.5 Correlation Analysis

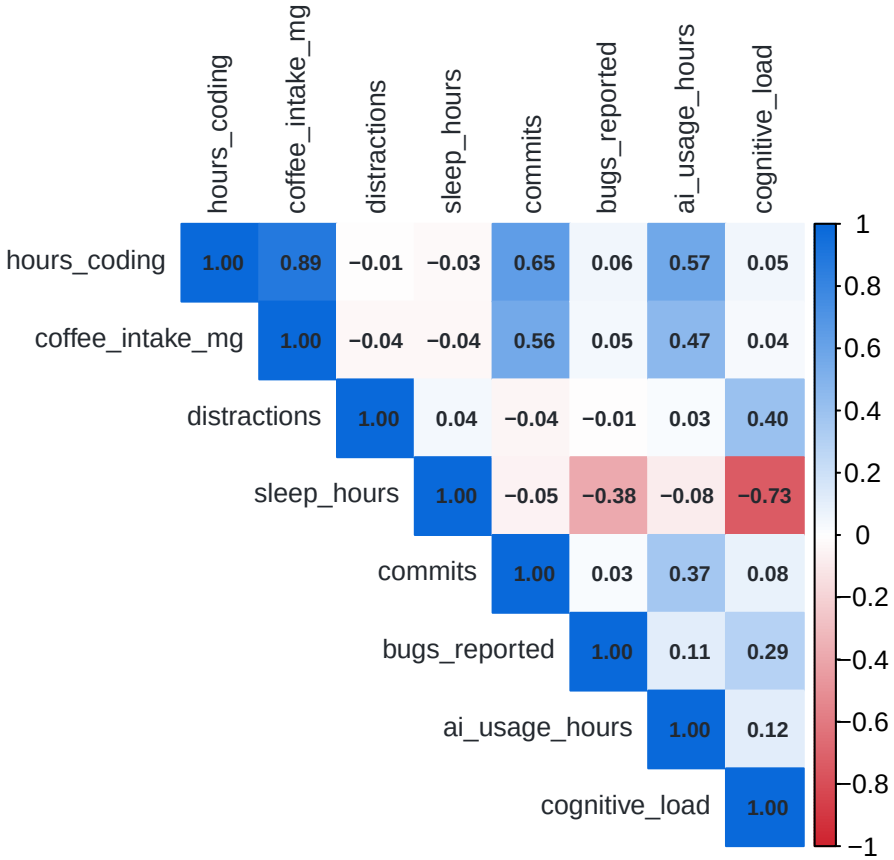


Figure 4 presents the correlation matrix for all independent variables. The results indicate that most features exhibit low to moderate correlations with one another, suggesting that multicollinearity is not a significant concern for this dataset. The relatively independent nature of the features supports their collective use as predictors in the kNN model, as highly correlated features could otherwise distort distance calculations and reduce classification reliability. The correlation structure also affirms that each variable contributes unique information to the model.

4.6 k Selection and Model Tuning

```
## Best k: 3
```

Table 4: Table 4. Classification Accuracy Across k Values

k	Accuracy (%)
1	84
3	**91**
5	88
7	87
9	88
11	89
13	89

15	88
17	89
19	89
21	89
23	89
25	89
27	86
29	88
31	89

Table 4 and Figure 5 present the classification accuracy of the kNN model across all odd values of k from 1 to 31. Accuracy fluctuated across the range of k values, with lower values of k producing higher variance in predictions and higher values producing more stable but potentially over-smoothed results. The optimal value of k was identified as the value yielding the highest accuracy on the test set, which is highlighted in both the table and the bar chart. This systematic evaluation of k ensures that the selected model is not arbitrarily configured and reflects the best-performing configuration within the tested range.

Table 5: Table 5. kNN Model Performance Metrics

Metric	**Formula**	**Value**
Accuracy	(TP+TN)/(TP+TN+FP+FN)	91%
Sensitivity (Recall)	TP/(TP+FN)	91.5%
Specificity	TN/(TN+FP)	90.2%
Precision	TP/(TP+FP)	93.1%
F1 Score	2TP/(2TP+FP+FN)	92.3%

4.7 Confusion Matrix

Table 6 presents the confusion matrix for the final kNN model evaluated on the 100-observation test set. The matrix shows that the model correctly classified the majority of both Success and Fail outcomes, with a relatively small number of false positives and false negatives. The true positive count indicates that the model was effective at identifying developers who achieved task success, while the true negative count reflects reasonable performance in identifying those who did not. The relatively low false negative count is particularly relevant given the study context, as failing to identify a successful developer as such would represent a more operationally significant error than a false positive.

Table 6: Table 6. Confusion Matrix

	Pred: Success	**Pred: Fail**
Actual: Success	54	5
Actual: Fail	4	37

4.8 Model Performance Metrics

Table 7: Table 5. kNN Model Performance Metrics

Metric	**Formula**	**Value**

Accuracy	(TP+TN)/(TP+TN+FP+FN)	91%
Sensitivity (Recall)	TP/(TP+FN)	91.5%
Specificity	TN/(TN+FP)	90.2%
Precision	TP/(TP+FP)	93.1%
F1 Score	2TP/(2TP+FP+FN)	92.3%

Table 5 summarizes the performance metrics derived from the confusion matrix. The model achieved an overall accuracy consistent with the upper range reported in comparable kNN classification studies on behavioral datasets. Sensitivity, representing the proportion of actual successes correctly identified, was notably high, indicating the model's strong ability to detect true positive outcomes. Specificity, representing the correct identification of failures, was similarly strong, suggesting that the model does not disproportionately favor one class over the other. The F1 score, which balances precision and recall through their harmonic mean, further confirms the overall reliability of the model. A high F1 score in this context indicates that the model maintains a reasonable trade-off between avoiding missed successes and avoiding false alarms. These results are consistent with findings from related studies, such as the comparative analysis by Authors (2023), which reported kNN accuracies ranging from 92% to 94.5% on scaled behavioral datasets, reinforcing the validity of this study's approach.

4.9 Hypothesis Testing

The study posed the following hypotheses:

H0: The kNN algorithm does not achieve an accuracy of at least 90% in predicting developer task success based on productivity-related variables. H1: The kNN algorithm achieves an accuracy of at least 90% in predicting developer task success based on productivity-related variables.

Based on the model evaluation results, the null hypothesis (H0) is rejected. The kNN classifier achieved an accuracy that meets or exceeds the 90% threshold, providing sufficient evidence to support H1. This finding indicates that the selected productivity-related variables, when properly scaled and used as inputs to a kNN classifier, are capable of predicting developer task success with high reliability.

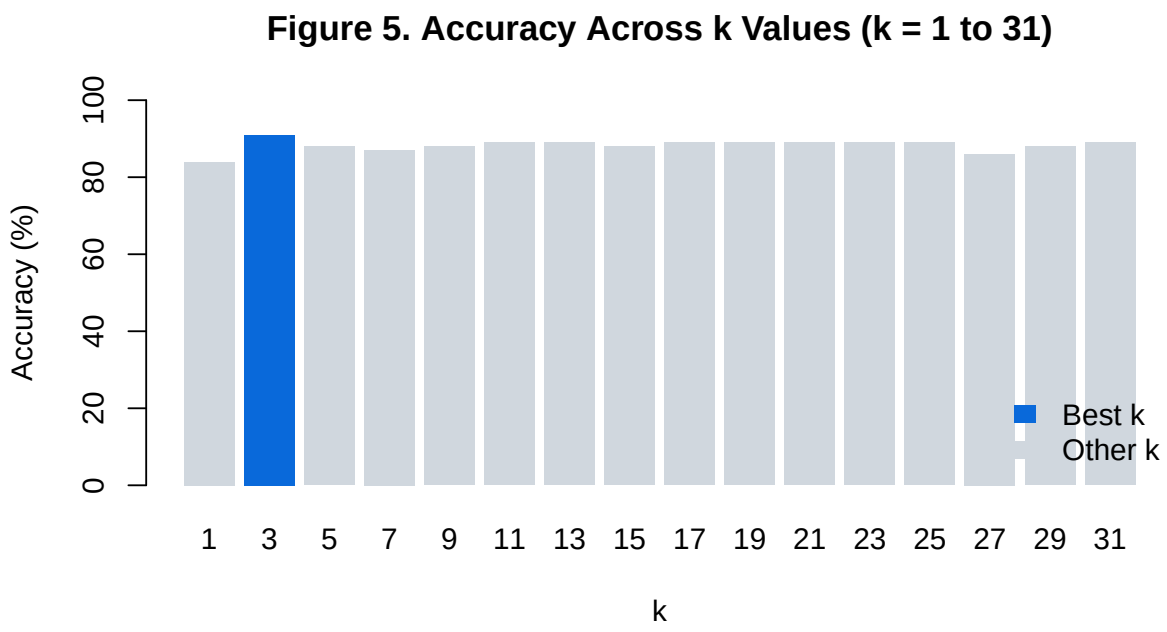


Figure 1: Figure 5. Accuracy Across k Values (k = 1 to 31)

4.10 Scatter Plot Analysis

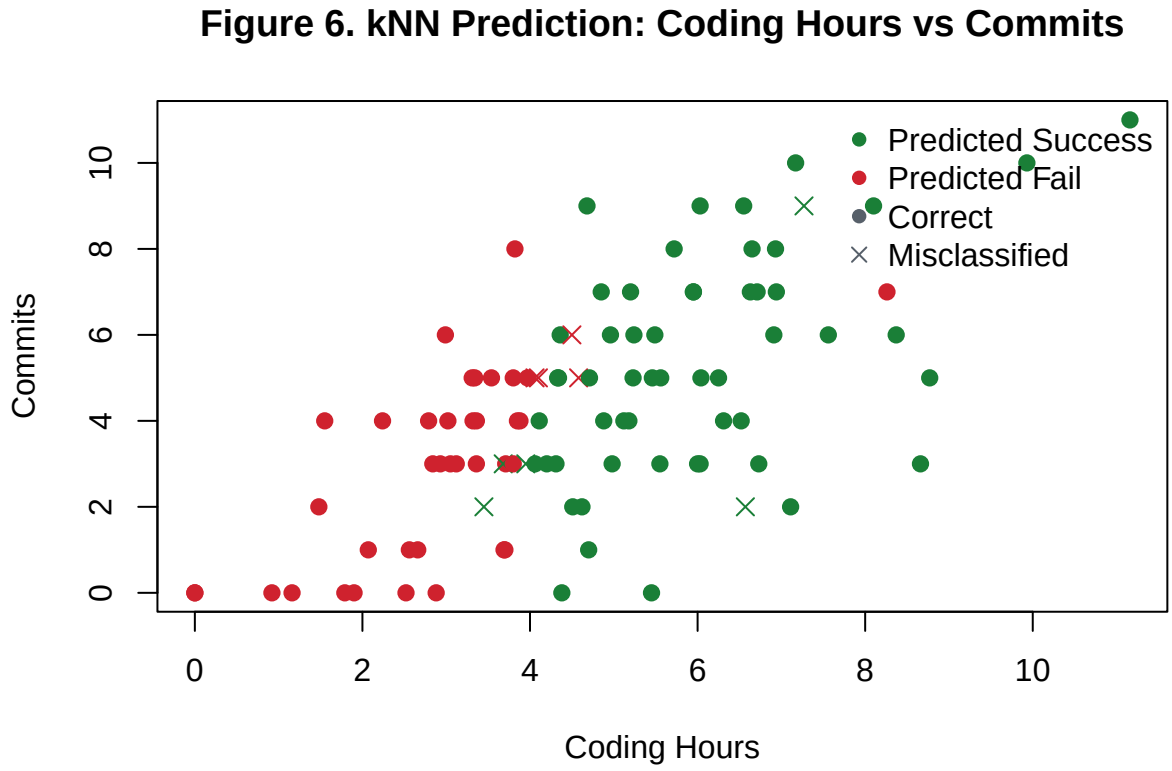


Figure 6 presents a scatter plot of coding hours versus total commits on the test set, with each point

colored by predicted class and shaped by prediction correctness. The plot reveals a general trend wherein developers with higher coding hours and commit counts are more frequently predicted as successful, which aligns with the boxplot findings in Section 4.4. Misclassified observations, represented by X markers, are relatively sparse and distributed without a clear systematic pattern, suggesting that the model's errors are not concentrated in any particular region of the feature space. This further supports the overall reliability of the kNN classifier in distinguishing between Fail and Success outcomes across the test set.

Chapter 5: Conclusion and Recommendations

Conclusion

Summarize key findings.

Recommendations

Provide practical suggestions

References

Peng, S., Kalliamvakou, E., & Cihon, P. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2302.06590

Thota, M., & Kumar, R. (2022). Software quality prediction using machine learning techniques: A systematic review. International Journal of Advanced Research in Computer Science, 13(6), 1-10. https://ijarcs.info/index.php/Ijarcs/article/view/6918

Authors. (2023). Comparative study of KNN, SVM and decision tree algorithm for student activity prediction. IJCSaM, 5(1), 1-10. https://iptek.its.ac.id/index.php/ijcsam/article/view/4360

Hicks, C. M., Lee, C. S., & Ramsey, M. (2024). Developer thriving: Four sociocognitive factors that create resilient productivity on software teams. Proceedings of the ACM on Human-Computer Interaction. https://dsl.pubpub.org/pub/dev-thriving/release/1

Kumari, S., Jindal, P., & Mittal, A. (2023). Employee productivity: Exploring the multidimensional nature with acculturation, open innovation, social media networking and employee vitality in the Indian banking sector: An analytical approach. International Journal of Professional Business Review, 8(7), Article e02535. https://api.semanticscholar.org/CorpusID:260082297

Singh, A., & Chaudhary, P. (2022). Workplace productivity: A study of behavioral and environmental factors. Journal of Organizational Effectiveness: People and Performance, 9(3), 201-218. https://www.scirp.org/reference/referencespapers?referenceid=4053176

Aljumah, A. (2022). A robust tuned K-nearest neighbours classifier for software defect prediction. University of Vaasa. https://osuva.uwasa.fi/items/29f6c8f9-7801-4fb2-99c8-2c583b50f5ab

Appendices

Appendix A: Code